

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

TUIA NLP 2025



Trabajo Final: RAG + Agente ReAct

Alumno: Alejandro Armas

Docentes:

- Alan Geary
- Juan Pablo Manson
- Constantino Ferrucci

Fuentes de información

Distribución de las fuentes de información entre los tipos de fuentes de datos.

Tipo de Base de Datos	Fuentes utilizadas
Vectorial (RAG)	manuales_productos, resenas_usuarios, faqs.json
Tabular (Pandas)	ventas_historicas.csv, productos.csv, tickets_soporte.csv, inventario_sucursales.csv
Grafo (Neo4j)	ventas_historicas.csv, productos.csv, tickets_soporte.csv, vendedores.csv, devoluciones.csv

Los manuales de los productos son archivos `.md` que mantienen una estructura común, como indicar *Marca* y *Modelo* dentro del primer y único `#header` de nivel 1. Extraigo dicha información con expresiones regulares para poder inyectarla como contexto y metadata (mejorando los embeddings). También usé un splitter de markdown, separando en cada *sección* y *subsección* (headers de nivel 2 y 3) de cada manual. Luego proceso esos splits con otro splitter para limitar los chunks a 500 caracteres.

Las reseñas de los productos son archivos `.txt` que también mantienen una estructura común, donde primero indican un set de metadatos y luego el texto crudo de la reseña. De nuevo extraigo los datos de utilidad mediante expresiones regulares para poder utilizarlos como contexto y metadata.

El archivo `faqs.json` es fácilmente procesable, teniendo explicitada la respuesta a cada pregunta y la metadata del producto. También lo proceso adecuadamente para generar buenos embeddings.

Los demás son archivos `.csv` que distribuyo entre la base de datos tabular y la de grafos, según la naturaleza de la información.

Por ejemplo para la base de grafos incluyo `devoluciones.csv` y `ventas_historicas.csv`, permitiendo modelar que un *nodo usuario* **compro** o **devolvió** un *nodo producto*.

Y para la base tabular incluyo `inventario_sucursales.csv` que contiene información estructurada cuantitativa, perfecta para modelar tabularmente.

Modelos utilizados

LLM: Gemini 2.5 Flash (cloud)

Elegido por su buen desempeño general, velocidad y accesibilidad mediante api. Lo utilizo para:

- Generar prompts de consulta para la fuente de datos tabular (pandas)
- Generar prompts de consulta para la fuente de datos de grafos (cypher)
- Clasificador de intención para el RAG. (Métricas perfectas)
- "Cerebro" del Agente ReAct. Interpreta la query del usuario, genera los prompts para cada herramienta y genera la respuesta final.

Embeddings: gemma-300m (local)

Elegido por su excelente desempeño con una baja cantidad de parametros (300 millones), lo que permite ejecutarse rápido en el entorno de colab (GPU T4) y por estar entrenado en el idioma español. Utilizado para generar embeddings de manuales, reseñas y preguntas frecuentes, permitiendo la búsqueda semántica.

Reranker: mmarco-mMiniLMv2-L12-H384 (local)

Elegido por ser el mejor modelo disponible en huggingface soporte soporte de idioma español, además al ser un modelo chico corre bien en el entorno de colab (GPU T4). Utilizado para implementar la búsqueda híbrida sobre la fuente de datos vectorial (Chroma), sirve como un filtro de ruido final, asegurandose de devolverle al llm los fragmentos realmente relevantes. (Agente ReAct)

Resultados de ejecución

RAG

Desenpeño

```
print(rag.responder("¿Cómo uso mi licuadora para hacer smoothies?"))
```

Python

¡Hola! Para usar tu licuadora Compacto de ChefMaster (modelo P0004) para preparar un smoothie de frutas, sigue estos sencillos pasos:

1. ****Prepara las frutas:**** Lava y corta las frutas en trozos medianos (aproximadamente de 2 a 3 cm).
2. ****Agrega los líquidos:**** Coloca primero los ingredientes líquidos en la jarra de la licuadora (como leche, yogurt o jugo).
3. ****Añade las frutas y el hielo:**** Luego, agrega las frutas cortadas y el hielo.
4. ****Cierra la tapa:**** Asegúrate de que la tapa esté bien cerrada y sellada herméticamente.
5. ****Inicia el licuado:**** Comienza a licuar a velocidad baja (nivel 1-2) durante unos 10 segundos.
6. ****Aumenta la velocidad:**** Gradualmente, aumenta la velocidad a un nivel alto (nivel 4-5) hasta obtener la consistencia deseada.

¡Así de fácil tendrás tu smoothie listo!

```
print(rag.responder("¿Cuáles son las licuadoras de menos de $200?"))
```

Python

Las licuadoras con un precio menor a \$200 son:

- * Licuadora Basic (TechHome) - \$150.00
- * Compacto Licuadora (HomeChef) - \$180.00
- * Licuadora Viajera (TechHome) - \$120.00

```
print(rag.responder("Me compré la cafetera P0121, que voltaje necesita?"))
```

Python

La cafetera P0121 necesita un voltaje de 12V.

```
print(rag.responder("¿Qué opinan los usuarios de esta cafetera?"))
```

Python

Lo siento, no tengo opiniones de usuarios para la cafetera P0121. La información que he encontrado corresponde al modelo P0122.

La última pregunta demostró el buen desempeño del RAG, por un lado entendió que le estaba haciendo otra pregunta sobre la P0121 por historial, y por el otro me respondió honestamente que no tiene reseñas de usuarios sobre dicho modelo.

```
print(rag.responder("Quiero una licuadora con buenas reseñas"))
```

Python

Lo siento, no he encontrado licuadoras con reseñas que se consideren "buenas" en la información disponible. Las reseñas que encontré para los modelos P0001, P0005 y P0006 indican que tienen "sus pros y contras" y que

```
print(rag.responder("¿Qué voltaje requiere el rallador digital eléctrico?"))
```

Python

El rallador digital eléctrico requiere un voltaje de 110V.

En este caso habría sido mejor que nos pregunte que modelo de rallador específico tenemos.

```
print(rag.responder("Muéstrame 10 productos de la categoría Cocina"))
```

Python

¡Claro! Aquí tienes 10 productos de la categoría Cocina que tenemos disponibles:

- * Licuadora
- * Plus licuadora Pro
- * Compacto licuadora
- * Ultra licuadora
- * Procesadora
- * Horno Eléctrico
- * Batidora de Mano
- * Tostadora
- * Cafetera
- * Microondas

¿Hay algún electrodoméstico de cocina en particular que te interese, o tienes alguna otra consulta?

```
print(rag.responder("¿Qué usuarios devolvieron todos los productos que compraron?"))
```

Python

¡Hola! Los usuarios que devolvieron todos los productos que compraron son: Lucas Ramírez, Gabriel Benítez, Julieta Silva, Agustina Flores y Rodrigo Rivera.

Agente ReAct

```
# Prueba de multiples consultas en un mismo prompt, activo steps para ver el razonamiento completo

query = "¿Cuál es el precio del producto modelo P0005 y qué dicen las reseñas sobre él?, además dame el nombre de los usuarios que la compraron y la devolvieron"
consultar_agente(query, steps=True)
```

Python

Paso 1

Pensamiento: Question: ¿Cuál es el precio del producto modelo P0005 y qué dicen las reseñas sobre él?, además dame el nombre de los usuarios que la compraron y la devolvieron
Thought: El usuario quiere saber el precio, las reseñas, y los usuarios que compraron y devolvieron el producto P0005.
Primero, usaré 'table_search' para obtener el precio del producto P0005.
Action: table_search
Action Input: precio del producto P0005

Herramienta: table_search

Input: precio del producto P0005

Resultado: 2602.78

Paso 2

Pensamiento: Ahora tengo el precio del producto P0005. El siguiente paso es obtener las reseñas y los usuarios que compraron y devolvieron el producto.
Para las reseñas, usaré 'doc_search' .
Action: doc_search
Action Input: reseñas del producto P0005

Herramienta: doc_search

Input: reseñas del producto P0005

...

.....

El precio del producto modelo P0005 es \$2602.78. Las reseñas sobre este producto son variadas: algunos usuarios lo consideran de calidad normal, sin sorpresas y con pros y contras por el precio, mientras que otro us

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

El agente identificó correctamente los 3 pedidos y la herramienta correcta para cada caso, y formulo una respuesta bien formateada con toda la información recolectada.

El precio del producto modelo P0005 es \$2602.78. Las reseñas sobre este producto son variadas: algunos usuarios lo consideran de calidad normal, sin sorpresas y con pros y contras por el precio, mientras que otro usuario está muy feliz con la calidad, lo describe como preciso y duradero, y lo recomienda. Los usuarios que compraron y devolvieron el producto P0005 son Sofía Cruz, Emma Torres y Lucas Benítez.

El agente identificó correctamente los 3 pedidos y la herramienta correcta para cada caso, y formulo una respuesta bien formateada con toda la información recolectada.

```
# Prueba de contexto e historial
consultar_agente("Que dice la peor reseña?", steps=True)
```

Python

Paso 1

Pensamiento: Thought: El usuario quiere saber el contenido de la "peor reseña" para el producto P0005. En el historial ya se mencionaron reseñas variadas, incluyendo algunas no tan positivas. Necesito usar 'doc_sea

Action: doc_search

Action Input: ¿Cuál es la peor reseña del producto modelo P0005?

Herramienta: doc_search

Input: ¿Cuál es la peor reseña del producto modelo P0005?

Resultado: Fuente: reseña - Licuadora

Reseña de Producto: Licuadora Modelo: P0005

OPINIÓN:

Buenas! Tiene sus pros y contras con Licuadora. Tiene cosas buenas y cosas malas. Por el precio está bien, pero no esperen maravillas. Espero les sirva!

Fuente: reseña - Licuadora

Reseña de Producto: Licuadora Modelo: P0005

OPINIÓN:

Hola! Lamentablemente debo decir que la calidad es muy mala con Licuadora. Es muy lento y difícil de limpiar. Se rompió a los pocos días de uso. Espero les sirva!

Fuente: reseña - Licuadora

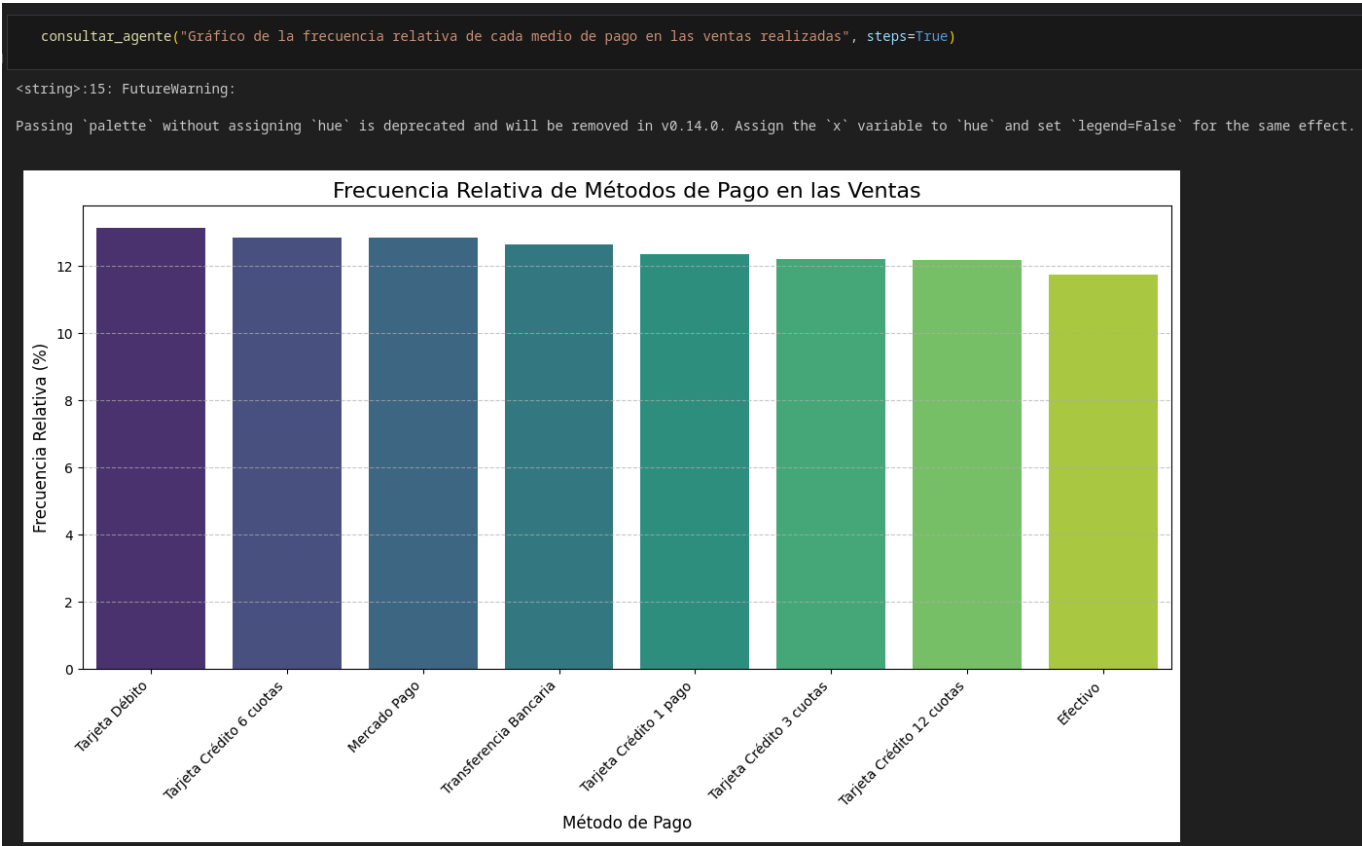
Reseña de Producto: Licuadora Modelo: P0005

...

La peor reseña sobre el producto modelo P0005 dice: "Lamentablemente debo decir que la calidad es muy mala con Licuadora. Es muy lento y difícil de limpiar. Se rompió a los pocos días de uso. Espero les sirva!"

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

El Agente comprendió lo que le estaba preguntando correctamente entendiendo el historial y el contexto de la conversación.



```
Paso 1

Pensamiento: Action: analytics_tool
Action Input: Graficar la frecuencia relativa de cada medio de pago en las ventas realizadas.

Herramienta: analytics_tool

Input: Graficar la frecuencia relativa de cada medio de pago en las ventas realizadas.

Resultado: Gráfico generado correctamente.
generado mediante
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

resultado = ""

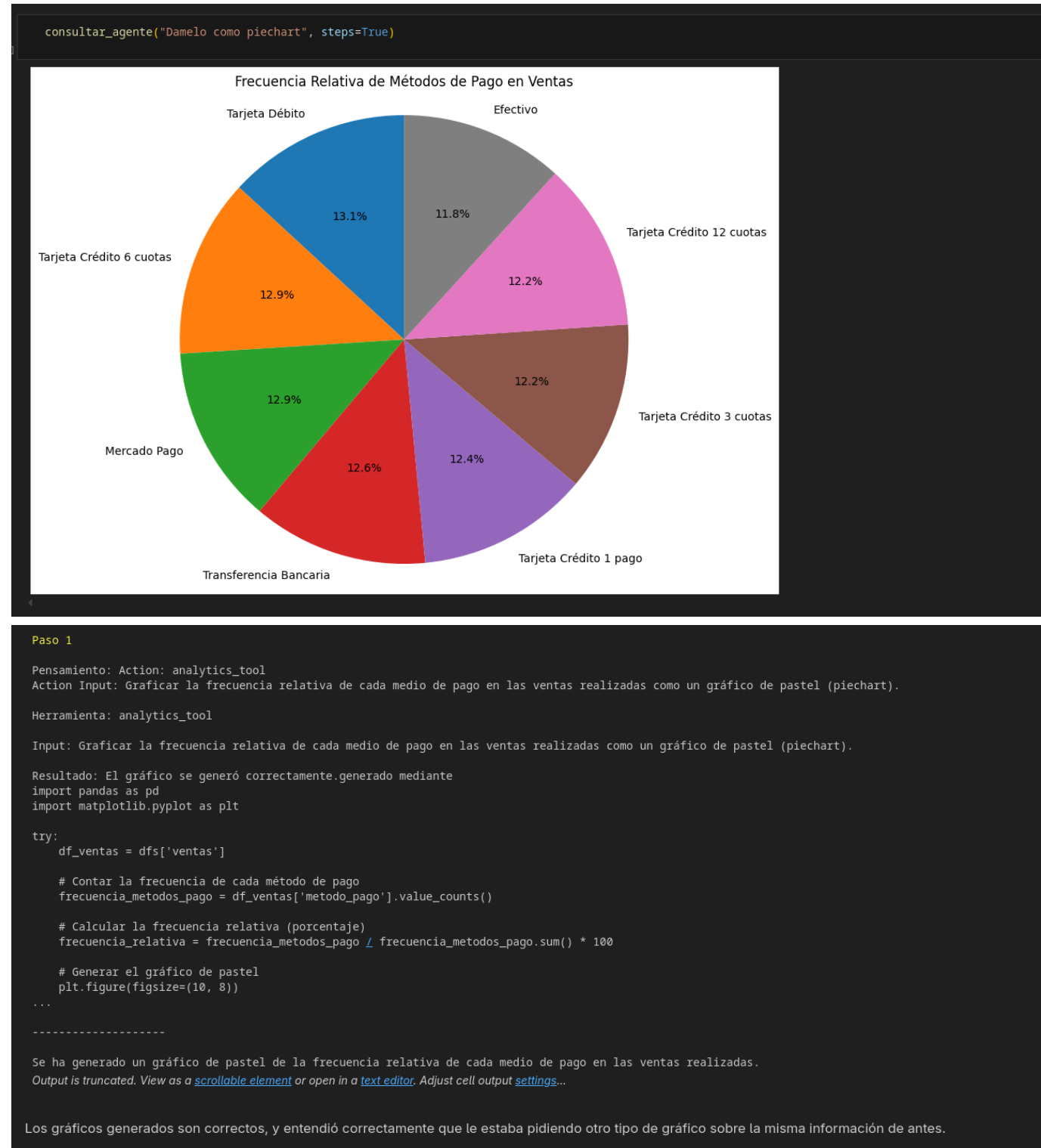
try:
    df_ventas = dfs['ventas']

    # Calcular la frecuencia relativa de cada método de pago
    frecuencia_relativa = df_ventas['metodo_pago'].value_counts(normalize=True) * 100

    # Crear el gráfico de barras
    plt.figure(figsize=(12, 7))
    ....

-----

Se ha generado un gráfico de la frecuencia relativa de cada medio de pago en las ventas realizadas.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



```
consultar_agente("Decime quienes son los usuarios que mas compraron, cuanto compraron y quien les vendió", steps=True)

Paso 1
Pensamiento: Action: graph_search
Action Input: usuarios que mas compraron, el monto total de sus compras y los vendedores involucrados.
Herramienta: graph_search
Input: usuarios que mas compraron, el monto total de sus compras y los vendedores involucrados.
Resultado: [{"Usuario": 'Ignacio Castillo', 'montoTotalCompras': 270833.36000000003, 'vendedoresInvolucrados': ['Laura Rodríguez', 'Carlos Rojas', 'Benjamín Suárez', 'Emilia Mendoza', 'Florencia Ríos', 'Jazmín Medina', 'Santiago Flores', 'Tomás Rome', 'Pablo Cruz', 'Renata García', 'Thiago Rodríguez', 'Benjamín Suárez', 'Bianca Suárez', 'Milagros Hernández', 'Santiago Flores', 'Maria Benit', 'Gonzalo Cabrera', 'Emanuel Navarro', 'Gonzalo Rios', 'Adrián Reyes', 'Martina Castillo', 'Facundo Fernández', 'Juan Díaz', 'Benjamín Álvarez', 'Laura Álvarez']}

Los 3 usuarios que más compraron son:
* **Ignacio Castillo**: Compró un total de $270833.36. Los vendedores involucrados fueron: Laura Rodríguez, Carlos Rojas, Benjamín Suárez, Emilia Mendoza, Florencia Ríos, Jazmín Medina, Santiago Flores, Tomás Rome
* **Pablo Cruz**: Compró un total de $249139.44. Los vendedores involucrados fueron: Carlos Flores, Renata García, Thiago Rodríguez, Benjamín Suárez, Bianca Suárez, Milagros Hernández, Santiago Flores, Maria Benit
* **Gonzalo Cabrera**: Compró un total de $242176.87. Los vendedores involucrados fueron: Emanuel Navarro, Gonzalo Rios, Adrián Reyes, Martina Castillo, Facundo Fernández, Juan Díaz, Benjamín Álvarez, Laura Álvarez

historial_agente.pop(-1) # Borro ultima interacción del historial

('Decime quienes son los usuarios que mas compraron, cuanto compraron y quien les vendió',
'Los 3 usuarios que más compraron son:\n'
**Ignacio Castillo**: Compró un total de $270833.36. Los vendedores involucrados fueron: Laura Rodríguez, Carlos Rojas, Benjamín Suárez, Emilia Mendoza, Florencia Ríos, Jazmín Medina, Santiago Flores, Tomás Rome, Pablo Cruz, Renata García, Thiago Rodríguez, Benjamín Suárez, Bianca Suárez, Milagros Hernández, Santiago Flores, Maria Benit, Gonzalo Cabrera, Emanuel Navarro, Gonzalo Rios, Adrián Reyes, Martina Castillo, Facundo Fernández, Juan Díaz, Benjamín Álvarez, Laura Álvarez)
```

```
consultar_agente("Decime quienes son los usuarios que mas compraron, cuanto compraron y quien les vendió", steps=True)

Paso 1
Pensamiento: Question: Decime quienes son los usuarios que mas compraron, cuanto compraron y quien les vendió
Thought: El usuario quiere saber quiénes son los usuarios que más compraron, cuánto compraron y quién les vendió. Esto implica buscar relaciones entre usuarios, ventas y vendedores, y también sumar las cantidades de
Action: graph_search
Action Input: Usuarios que mas compraron, el monto total de sus compras y los vendedores que les vendieron.
Herramienta: graph_search
Input: Usuarios que mas compraron, el monto total de sus compras y los vendedores que les vendieron.
Resultado: [{"Usuario": 'Ignacio Castillo', 'MontoTotalCompras': 22582.780000000002, 'VendedoresQueAtendieron': ['Laura', 'Carlos', 'Benjamín', 'Emilia', 'Florencia', 'Jazmín', 'Santiago', 'Tomás', 'Joaquín', 'Ignacio', 'Maria Gutierrez', 'Aana Castillo', 'Pablo Cruz', 'Francisco Romero', 'Emanuel', 'Lorenzo', 'Thiago', 'Julieta', 'Pablo', 'Miranda', 'Lucia', 'Gabriel', 'Matias', 'Tomás', 'Joaquin', 'Renata', 'Bianca', 'Milagros', 'Santiago', 'Maria', 'Martina', 'Santiago Flores', 'Tomás Rome', 'Pablo Cruz', 'Renata García', 'Thiago Rodríguez', 'Benjamín Suárez', 'Bianca Suárez', 'Milagros Hernández', 'Santiago Flores', 'Maria Benit', 'Gonzalo Cabrera', 'Emanuel Navarro', 'Gonzalo Rios', 'Adrián Reyes', 'Martina Castillo', 'Facundo Fernández', 'Juan Díaz', 'Benjamín Álvarez', 'Laura Álvarez']}

Los usuarios que más compraron son:
* **Ignacio Castillo** con un monto total de compras de $22582.78, atendido por Laura, Carlos, Benjamín, Emilia, Florencia, Jazmin, Santiago, Tomás, Joaquin, Ignacio.
* **Maria Gutierrez** con un monto total de compras de $21978.18, atendido por Emanuel, Lorenzo, Martina, Pablo, Florencia, Tomás, Benjamín, Santiago, Jazmin.
* **Aana Castillo** con un monto total de compras de $21178.38, atendido por Emanuel, Adrián, Thiago, Julieta, Pablo, Miranda, Lucia, Gabriel, Matias, Tomás, Joaquin.
* **Pablo Cruz** con un monto total de compras de $20761.62, atendido por Carlos, Renata, Thiago, Benjamín, Bianca, Milagros, Santiago, Maria, Martina.
* **Francisco Romero** con un monto total de compras de $20556.98, atendido por Julieta, Pablo, Florencia, Gabriel, Thiago, Adrián, Ignacio.
```

Los montos totales calculados son inconsistentes, distintas ejecuciones limpias (borrando el historial de interacciones) producen resultados diferentes.

Más ejemplos ejecutados disponibles en el notebook [RAG_Agente_ReAct.ipynb](#)

Posibles mejoras

- Forzar al agente a usar `table_search()` cuando necesite calcular un número.
- Memoria persistente guardando el historial de interacciones en una base de datos.
- Implementar respuesta por tokens (Máquina de escribir) para reducir ansiedad.